

Competitive Programming

Section 59 Editorial: ICPC Team Strategy

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
1.1	Detailed solution guide	3
2	A. Team Assignment	4
2.1	Restatement	4
2.2	Observations and algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	4
3	B. Team Scoreboard	6
3.1	Restatement	6
3.2	Observations and algorithm	6
3.3	Correctness proof	6
3.4	Complexity	6
3.5	Full C++ solution	6
3.6	Full Python solution	6
4	C. Template Fibonacci	8
4.1	Restatement	8
4.2	Observations and algorithm	8
4.3	Correctness proof	8
4.4	Complexity	8
4.5	Full C++ solution	8
4.6	Full Python solution	8
5	What to carry forward	10

1 Editorial overview

1.1 Detailed solution guide

A. Team Assignment Planning. The local task is a partition-style planning problem. Represent each problem's expected effort or value as an item, then use subset DP to find a balanced split. The invariant is that after processing the first i items, the reachable set contains exactly the sums achievable by choosing a subset of those items.

B. Scoreboard Simulation. ICPC ranking is deterministic once solved counts and penalties are known. Accumulate each team's solved count and penalty, then sort by the official order: more solved, less penalty, then name or id as the statement specifies. Keeping the comparator explicit avoids hidden tie bugs.

C. Template Recall. The Fibonacci/template exercise checks whether the team can deploy a known logarithmic routine under time pressure. Fast doubling keeps $(F(k), F(k+1))$; each binary digit doubles the index and optionally advances by one. This is the same invariant used in the advanced mixed contest.

Common pitfalls. Strategy exercises should still have proofs. The proof is usually that the simulated rule, DP invariant, or template invariant is exactly the contest rule being asked for.

The problems model team operations with algorithms already taught.

- A balances implementation workload by subset partition.
- B applies contest ranking rules, useful for avoiding scoreboard interpretation mistakes.
- C is a trusted notebook drill: fast Fibonacci by doubling.

The programming content is deliberately familiar; the strategic lesson is to make known templates fast, shared, and reliable under ICPC constraints.

2 A. Team Assignment

2.1 Restatement

Split implementation workload between two people minimizing max time.

2.2 Observations and algorithm

Use the invariant named in the editorial overview. The code processes the input in the natural order and updates exactly the state described by that invariant, so each branch corresponds to one mathematical case rather than an implementation trick.

2.3 Correctness proof

Correctness follows from the invariant proved in the detailed guide for this problem. The code initializes the base state, applies exactly the transition described there, and prints the value represented by the final state. $\square$

2.4 Complexity

The complexity follows from the concrete loops and data structures in the code below; the detailed guide also indicates which optimized structure is needed for larger external versions.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;if(!(cin>
```

2.6 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n=data[0]; a=data[1:]; best=10**9
    for mask in range(1<=n):
```

```
s=sum(a[i] for i in range(n) if mask>>i&1); best=min(best,max(s,sum(a)-s))
print(best)
if __name__=="__main__": main()
```

3 B. Team Scoreboard

3.1 Restatement

Rank teams by contest rules.

3.2 Observations and algorithm

Use the invariant named in the editorial overview. The code processes the input in the natural order and updates exactly the state described by that invariant, so each branch corresponds to one mathematical case rather than an implementation trick.

3.3 Correctness proof

Correctness follows from the invariant proved in the detailed guide for this problem. The code initializes the base state, applies exactly the transition described there, and prints the value represented by the final state. $\square$

3.4 Complexity

The complexity follows from the concrete loops and data structures in the code below; the detailed guide also indicates which optimized structure is needed for larger external versions.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;if(!(cin>
```

3.6 Full Python solution

```
import sys
def main():
    lines=sys.stdin.read().splitlines()
    if not lines: return
    rows=[]
    for line in lines[1:]:
```

```
        name,sol,pen=line.split(); rows.append((-int(sol),int(pen),name))
    rows.sort(); print("\n".join(name for _,_,name in rows))
if __name__=="__main__": main()
```

4 C. Template Fibonacci

4.1 Restatement

Fast recurrence query drill for the team notebook.

4.2 Observations and algorithm

Use the invariant named in the editorial overview. The code processes the input in the natural order and updates exactly the state described by that invariant, so each branch corresponds to one mathematical case rather than an implementation trick.

4.3 Correctness proof

Correctness follows from the invariant proved in the detailed guide for this problem. The code initializes the base state, applies exactly the transition described there, and prints the value represented by the final state. $\square$

4.4 Complexity

The complexity follows from the concrete loops and data structures in the code below; the detailed guide also indicates which optimized structure is needed for larger external versions.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; const long long MOD=1000000007; pair<long long,long long> fib(long long
```

4.6 Full Python solution

```
import sys
MOD=1_000_000_007
def fib(n):
    if n==0: return (0,1)
    a,b=fib(n//2)
    c=a*((2*b-a)%MOD)%MOD
```

```
    d=(a*a+b*b)%MOD
    return (d,(c+d)%MOD) if n%2 else (c,d)
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    print("\n".join(str(fib(x)[0]) for x in data[1:1+data[0]]))
if __name__=="__main__": main()
```

5 What to carry forward

Red-level improvement comes from making the invariant explicit, stress-testing it, and then upsolving the stronger external versions.